

# 02\_Python\_Operators

September 1, 2026

## 0.1 Python Operators

**Name: Vamshi / Date: 11/08/2026** In Python, Operators are used to perform operations on values and variables. The different types of operators are:

01. Arithmetic Operator
  02. Assignment Operator
  03. Relational Operator
  04. Logical Operator
  05. Bit-Wise Operator
  06. Ternary Operator
  07. Identity Operator
  08. Membership Operator
  09. Unary Operator
- 

### 0.1.1 01. Arithmetic Operator

Used to perform basic mathematical operations (+, -, \*, /, //, %, \*\*).

```
[1]: x = 21
     y = 4

     print("Addition\t\t", x + y)          # Addition (+)
     print("Subtraction\t\t", x - y)        # Subtraction (-)
     print("Multiplication\t\t", x * y)      # Multiplication (*)
     print("Division\t\t", x / y)           # Division (/)
     print("Floor Division\t\t", x//y)       # Floor Division (//)
     print("Mod\t\t\t", x % y)              # Modulus - returns remainder (%)

     a = 3
     b = 3
     print("a to power b\t\t", a ** b)      # power (**) /---> 3 to power 3 is 27 /--->
     ↪ 2 to power 4 is 16

     # here '/' is an escape sequence and is used to insert horizontal tab space.
```

|             |    |
|-------------|----|
| Addition    | 25 |
| Subtraction | 17 |

|                |      |
|----------------|------|
| Multiplication | 84   |
| Division       | 5.25 |
| Floor Division | 5    |
| Mod            | 1    |
| a to power b   | 27   |

---

### 0.1.2 02. Assignment Operator

Used to assign values to variables and update them.

```
[2]: # Assignment Operator (=)
```

```
a = 10
b = 20
c = a + b
print(c)
```

30

```
[3]: # Addition Assignment Operator (+=)
```

```
a1 = 10
a1 += 40 # It means a1 = a1 + 40 and result stored in a1
print(a1)

b1 = 50
b1 += a1 # It means b1 = b1 + a and result stored in b1
print(b1)
```

50

100

```
[4]: # Subtraction Assignment Operator (-=)
```

```
a2 = 5
b2 = 4
a2 -= b2 # It means a2 = a2 - b2 and result stored in a2
print(a2)
```

1

```
[5]: # Multiplication Assignment Operator (*=)
```

```
a3 = 10
a3 *= 50
print(a3)

b3 = 10
```

```
a3 *= b3
print(a3)
```

500  
5000

[6]: *# Division Assignment Operator (/=)*

```
x = 20
y = 5

x /= y
print(x)

x1 = 5
x1 /= 2
print(x1)
```

4.0  
2.5

[7]: *# Modulus Assignment Operator (%=)*

```
x2 = 5
y2 = 3

x2 %= y2
print(x2)

# For non-negative integers, if the left number is smaller than the right
↪ number, left % right = left.

x3 = 3
y3 = 5

x3 %= y3 # x3 is smaller than y3 ---> above statement is true
print(x3)
```

2  
3

The same concept is used for floor division (`//=`), exponent (`**=`), and with bitwise operators as well.

---

### 0.1.3 03. Relational Operator

Relational operators are also known as comparison operators. Used to compare values and returns the output as True or False based on the condition.

```
[8]: x4 = 25
      x5 = 60

      print(x4>x5)
      print(x4<x5)
      print(x4!=x5) # Not Equal
      print(x4==x5)
      print(x4>=x5)
      print(x4<=x5)
```

```
False
True
True
False
False
True
```

```
[9]: y4 = y5 = 300
      print(y4==y5)
```

```
True
```

---

#### 0.1.4 04. Logical Operator

Used to perform logical AND, logical OR, and logical NOT operations. They are combined with conditions.

```
[10]: i = True
      j = False

      print(i and j) # and - Output is True is both are True
      print(i or j) # or - Output is False only is both are False
      print(not j) # not - complement ---> True becomes False and False becomes True
```

```
False
True
True
```

```
[11]: i1 = 20
      j1 = 22

      print(i1>j1 or i1==j1)
      print(i1==20 and j1>i1)
```

```
False
True
```

---

### 0.1.5 05. Bitwise Operator

Used to perform bit operations on binary numbers.

- Bitwise AND (&)
- Bitwise OR (|)
- Bitwise NOT (~)
- Bitwise XOR (^)
- Bitwise Shift:
  - Left Shift («)
  - Right Shift (»)

```
[ ]: # Bitwise AND (&) operation, converts to binary and performs AND operation and
    ↪return the output.
```

```
a = 10      # binary = 1010
b = 2       # binary = 0010
r = a & b    # Result = 0010 ----> 2
print(r)
```

2

```
[15]: # Bitwise OR (|) operation, converts to binary and performs OR operation and
    ↪returns the output.
```

```
a = 10      # binary = 1010
b = 2       # binary = 0010
r = a | b    # Result = 1010 ----> 10
print(r)
```

10

```
[24]: # Bitwise NOT (~) operation, unary operator that returns the one's complement
    ↪of an integer by flipping all its bits (0 → 1, 1 → 0).
```

```
# The Rule: ~a = -(a + 1)
```

```
a = 8
r = ~a
print(r)
```

```
b = 10
r1 = ~b
print(r1)
```

```
c = 1
r2 = ~c
print(r2)
```

-9

-11  
-2

[17]: *# Bitwise XOR (^) operation, also known as the exclusive OR operator, is used  
↳ to perform the XOR operation on two operands and returns the output.*

```
a = 10      # binary = 1010
b = 2       # binary = 0010
r = a ^ b   # Result = 1000 ---> 8
print(r)
```

8

[ ]: *# Bitwise Shift operations, shift the bits of an integer left (<<) or right  
↳ (>>) by a specified number of positions.*

```
# Left Shift (<<) ---> Bits are gained
a = 12 #---> 1100
print(a<<2)    #---> 110000 <--- 2 bits are gained or shifted to left

# Right Shift (>>) ---> Bits are lost
b = 12 #---> 1100
print(b>>2)    #---> 0011 ---> 2 bits are lost or shifted to right
```

48  
3

---

### 0.1.6 06. Ternary Operator

[28]: *# also known as conditional expressions are operators that evaluate something  
↳ based on a condition being true or false*

```
a, b = 30, 50
min = a if a < b else b
print(min)
```

30

---

### 0.1.7 07. Identity Operator

“is” and “is not” are the identity operators, they are used to check if the values located in same memory location. It returns boolean value.

[ ]: *# Note: '==' and 'is' are not same. '==' checks if values are equal while 'is'  
↳ checks if values are in same MEMORY LOCATION.*

```

n1 = 12
n2 = 10

a = [2, 10, 31]
b = [15, 12, 31]
c = a

s1 = "hello world"
s2 = "hello world, I am Iron Man"
s3 = s2

print(n1 is n2)
print(a is b)
print(a is c, f"| id of a {id(a)}, id of c {id(c)}")
print(s1 is s2)
print(s3 is s2, f"| id of s2 {id(s2)}, id of s3 {id(s3)}")

print("-----")

# The "is not" operator checks if two variables point to different objects.
print(n1 is not n2)
print(a is not b)
print(a is not c, f"| id of a {id(a)}, id of c {id(c)}")
print(s1 is not s2)
print(s3 is not s2, f"| id of s2 {id(s2)}, id of s3 {id(s3)}")

```

```

False
False
True | id of a 1984711488704, id of c 1984711488704
False
True | id of s2 1984711632064, id of s3 1984711632064
-----
True
True
False | id of a 1984711488704, id of c 1984711488704
True
False | id of s2 1984711632064, id of s3 1984711632064

```

### 0.1.8 08. Membership Operator

They are used to check if a value or object is present in a sequence.

```

[ ]: # 'in' operator --> returns True if value is there in the sequence

a = [10, 30, 99, 1000, "Hulk"]
b = "Hi, You are Cute"
c = {1: "Apple", 2: "Banana", 3: "Cherry"}

```

```

d = (1, 3, 5, 7)

print("Hulk" in a)
print("we" in b)
print(3 in c) # dictionary checks keys not values
print(9 in d)

# Similarly, 'not in' works opposite to 'in' operator ---> returns True if
↳ value not found in sequence

```

True  
False  
True  
False

```

[37]: # Python also provides a function from the operator module called contains()
↳ that works like in.

# syntax- operator.contains(sequence, value)

import operator

print(operator.contains([1, 2, 3, 4, 5], 2))
print(operator.contains([1, 2, 3, 4, 5], 6))

```

True  
False

---

### 0.1.9 08. Unary Operator

```

[39]: # Unary Operator operates on only one operand

x = 5

print(+x)    # +5
print(-x)    # -5
print(~x)    # -6

y = True

print(not y) # False

```

5  
-5  
-6  
False

---